

Bài 11

Ngôn ngữ C **Cấu trúc**

Nội dung

- Cấu trúc
- Biến kiểu cấu trúc
- Tên kiểu của cấu trúc
- Các phép toán với cấu trúc
- Mảng và con trỏ cấu trúc
- Hàm với cấu trúc

Cấu trúc

- **Cấu trúc** là tập hợp **các biến không cùng kiểu** sử dụng **chung một tên**. Mỗi biến được gọi là một **thành viên** hoặc một **trường**.
 - Kiểu dữ liệu do người lập trình định nghĩa.
 - Thường để lưu các mục dữ liệu có liên hệ logic với nhau.
- Ví dụ: thông tin cầu thủ
 - Số căn cước
 - Họ tên
 - Chiều cao
 - Cân nặng
 - ...

Cấu trúc - Định nghĩa

- Lệnh định nghĩa cấu trúc:

```
struct <tên cấu trúc> {  
    <kiểu> <thành viên 1>;  
    <kiểu> <thành viên 2>;  
    ...  
    <kiểu> <thành viên n>;  
};
```
- Cấu trúc có thể được định nghĩa toàn cục hoặc cục bộ.
- <tên cấu trúc> cũng là một định danh.
- <thành viên> là riêng tư và có thể là một cấu trúc khác.

```
/* định nghĩa cấu trúc  
Player */  
struct Player {  
    char    idNumber[13];  
    char    *fullname;  
    int     height;  
    float   weight;  
};
```

Biến kiểu cấu trúc - Khai báo

- Lệnh khai báo biến kiểu cấu trúc:
struct <tên cấu trúc> <tên biến>;
- Biến kiểu cấu trúc cũng có thể được khai báo khi định nghĩa cấu trúc:

```
struct <tên cấu trúc> {  
    ...  
} <tên biến>;
```

Khai báo các biến cấu trúc Player p1, p2, p3

```
struct Player {  
    char    idNumber[13];  
    char    *fullname;  
    int      height;  
    float    weight;  
} p1, p2;  
  
struct Player p3;
```

Biến cấu trúc - Khởi tạo

- Lệnh khởi tạo giá trị cho các thành viên của một biến cấu trúc
struct <tên cấu trúc> <tên biến> = {<danh sách giá trị>;
• <danh sách giá trị> chứa các giá trị để khởi tạo cho các thành viên.

```
// khởi tạo đủ giá trị  
struct Player p3 = {"123456789012",  
                    "Paul Henry",  
                    185,  
                    85.5F};  
  
// khởi tạo thiếu giá trị  
struct Player p4 = {"123456789012",  
                    "Paul Henry"};
```

Với thành viên là mảng kiểu char giá trị khởi tạo phải là hằng chuỗi kí tự

Tên kiểu của cấu trúc

- **Tên kiểu của cấu trúc** giúp việc sử dụng cấu trúc trông tự nhiên hơn trong các khai báo.
 - Tên kiểu không thay thế cho định nghĩa của cấu trúc.
 - Lệnh typedef dùng để đặt tên kiểu cho một cấu trúc.
- Đặt tên kiểu khi định nghĩa cấu trúc

typedef struct {

...

} <tên kiểu của cấu trúc>;

hoặc

typedef struct <tên cấu trúc> {

...

} <tên kiểu của cấu trúc>;

Tên kiểu của cấu trúc - Ví dụ

```
// định nghĩa cấu trúc và đặt tên kiểu TypePlayer
typedef struct {
    char    idNumber[13];
    char    *fullname;
    int     height;
    float   weight;
} TypePlayer;

// khởi tạo đủ giá trị
TypePlayer p3 = {"123456789012",
                 "Paul Henry",
                 185,
                 85.5F};
```

Với tên kiểu TypePlayer, khi sử dụng không cần từ khóa struct và trông tự nhiên hơn

Các phép toán với cấu trúc

- **Phép toán dấu chấm .** để truy cập thành viên của một biến cấu trúc
<tên biến>.<thành viên>
- **Phép toán gán =** thực hiện sao chép nội dung của một biến cấu trúc vào một biến cấu trúc cùng loại.
- **Phép toán ép kiểu ()** thường dùng khi muốn gán nội dung cho biến cấu trúc từ danh sách các giá trị (***struct** <tên cấu trúc>*) hoặc
(*<tên kiểu của cấu trúc>*)

Các phép toán với cấu trúc - Ví dụ

```
TypePlayer p5, p6;  
// truy cập và gán giá trị các thành viên của p5  
strcpy(p5.idNumber, "123456789012");  
p5.fullname = "Paul Henry";  
p5.height = 185;  
p5.weight = 85.5F;  
  
/* ép danh sách giá trị thành một cấu trúc và sao  
chép cho p6 */  
p6 = (TypePlayer) {"123456789012",  
                  p5.fullname,  
                  p5.height - 5,  
                  85.5F};
```

Với thành viên là mảng kiểu char để lưu giá trị phải dùng hàm strcpy() thay vì phép gán

Mảng và con trỏ cấu trúc

- Khai báo mảng một chiều các cấu trúc
struct <tên cấu trúc> <tên biến>[kích thước];
hoặc
<tên kiểu của cấu trúc> <tên biến>[kích thước];
- Khai báo con trỏ cấu trúc
struct <tên cấu trúc> *<tên biến>;
hoặc
<tên kiểu của cấu trúc> *<tên biến>;
- **Phép toán mũi tên** -> để truy cập gián tiếp thành viên của một cấu trúc thông qua con trỏ cấu trúc
<con trỏ cấu trúc>-><thành viên>

Hàm với cấu trúc

```
typedef struct {  
    double real;  
    double imag;  
} Complex;  
  
// tính tổng 2 số phức  
Complex sum(Complex a, Complex b) {  
    Complex s;  
    s.real = a.real + b.real;  
    s.imag = a.imag + b.imag;  
    return s;  
}  
  
// cộng một số phức cho một số phức  
void add(Complex *a, Complex b) {  
    a->real = a->real + b.real;  
    a->imag = a->imag + b.imag;  
}
```